Laporan UTS Sistem Robot Otonom
Pemetaan 3D Menggunakan Drone dan LiDAR

Nama	Faizal Fitrohni
NRP	5022241019
Program studi	Teknik Elektro, Institut Teknologi Sepuluh Nopember
Mata kuliah	Sistem Robot Otonom
Dosen	Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
GitHub	https://github.com/faizalfit/faizal-drone-mapping

Video proses mapping: https://youtu.be/LP9T2kGMEIU?si=_66DVsWCc8ckNJEV
Video hasil mapping: https://youtu.be/frZXkwX_bZA?si=8PU8pDzxdcZ2fjby

1 Deskripsi Tugas

Tugas ini bertujuan membuat sistem pemetaan tiga dimensi menggunakan drone pada simulasi CoppeliaSim dan program Python. Sensor dipasang agar dapat mengamati permukaan tanah, sisi objek, dan bagian atas objek. Data hasil pengukuran dikumpulkan menjadi kumpulan titik tiga dimensi atau *point cloud*, lalu diolah dan ditampilkan agar bentuk lingkungan mudah diamati.

Penilaian dilakukan secara bertahap. Tahap C mencakup kontrol drone dan tampilan jalur, tahap BC mencakup scan 2D, tahap B mencakup scan 3D, tahap AB mencakup pengurangan jumlah titik dengan voxel, dan tahap A mencakup pewarnaan berdasarkan ketinggian. Sebagai tambahan, titik dikelompokkan berdasarkan kedekatan posisinya tanpa machine learning, dan kelompok tersebut dapat dipilih pada tampilan HTML. Pengumpulan tugas terdiri dari video proses dan hasil mapping, scene, program Python, script Lua, dan laporan PDF.

2 Desain dan Metode

2.1 Perangkat lunak, lingkungan, dan drone

Simulasi dijalankan di CoppeliaSim 4.10.0 pada Windows 11. Program Python berjalan pada environment Conda bernama sro_env yang berisi NumPy, Matplotlib, dan klien ZeroMQ Remote API. Scene berupa lantai berukuran 10×10 m dengan permukaan pada $Z = 0$. Objek tertinggi pada scene adalah pohon dengan tinggi sekitar 3,22108 m. Drone memakai model quadcopter bawaan yang bergerak mengikuti sebuah target posisi. Python menggeser target tersebut sedikit demi sedikit dan mencatat posisi drone yang sebenarnya.

2.2 Sensor LiDAR dan penempatannya

Sensor yang digunakan adalah Velodyne VLP-16 pada CoppeliaSim, yang di dalam simulator ditulis dengan ejaan VPL16. Sensor dipasang sebagai anak dari base drone dengan sudut beta sebesar 45° terhadap parent. Posisi ini dipilih agar badan drone tidak menghalangi pengukuran. Kemiringan tersebut membuat sebagian sapuan mengarah ke bawah sehingga tanah dan sisi objek dapat terbaca, sedangkan bagian puncak objek terbaca ketika drone melintas di atasnya.

Drone terbang pada ketinggian 5,2 m. Gabungan ketinggian, kemiringan sensor, serta perubahan posisi dan arah drone membuat objek teramati dari beberapa sudut. Meski demikian, tidak semua permukaan dapat terbaca karena bagian yang tertutup objek lain tetap menyisakan celah pada peta.

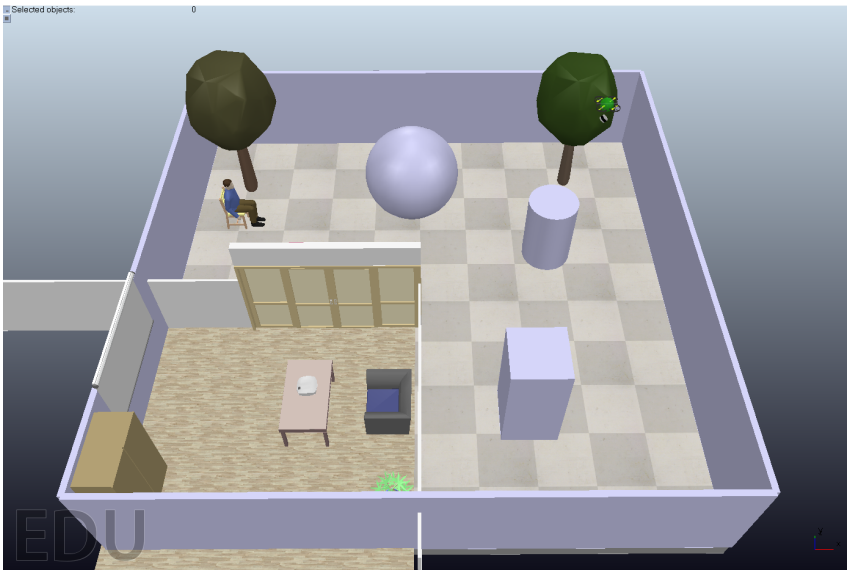


Figure 1: Scene pemetaan dengan lantai, objek, dan quadcopter.

Model sensor memiliki empat vision sensor di dalamnya, tetapi hanya dipakai plugin untuk mensimulasikan pengukuran LiDAR, bukan sebagai empat kamera yang menjadi sensor utama. Implementasi ini ditujukan untuk mempelajari pengolahan data LiDAR, sehingga penggunaan pada perangkat fisik masih memerlukan penyesuaian.

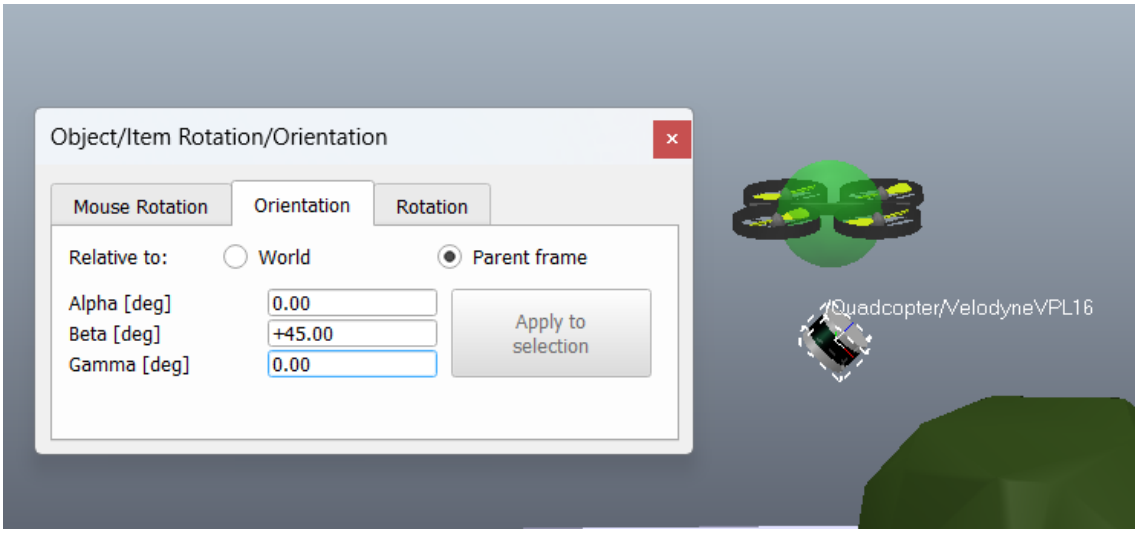


Figure 2: Penempatan LiDAR pada base dan orientasi beta 45 derajat terhadap parent.

2.3 Lintasan dan akumulasi titik

Drone melewati enam waypoint pada ketinggian 5,2 m, yaitu $(-4,-4)$, $(-4,4)$, $(0,4)$, $(0,-4)$, $(4,-4)$, dan $(4,4)$ dalam meter. Pola zigzag ini menyapu beberapa jalur yang sejajar. Gerak target dihaluskan agar awal dan akhir setiap perpindahan tidak mendadak. Kecepatan maksimum target diatur 0,5 m/s, sedangkan perubahan arah hadap (yaw) berlangsung selama 5 s.

Python dan simulasi dijalankan langkah demi langkah, sehingga setiap satu langkah simulasi langsung diikuti pembacaan data. Posisi drone dicatat sebagai trajectory. Titik LiDAR sudah diterima dalam koordinat dunia sehingga dapat langsung digabungkan dari waktu ke waktu. Artinya, posisi drone diambil dari simulator, bukan dari SLAM atau estimasi posisi sendiri. Titik dibatasi pada $|x|, |y| \leq 5{,}2$ m serta $-0{,}1 \leq z \leq 3{,}5$ m agar peta tidak melebar ke luar scene.

3 Script Lua Sensor

Script Lua dipasang pada model LiDAR dan berfungsi menghubungkan hasil sensor dengan Python. Penjelasan berikut mengacu pada script pengirim data dengan nama properti signal.faizal_lidar_packet_v2 yang dipakai selama pengujian. Nama properti ini harus sama dengan yang dibaca oleh program 01.

1. sysCall_init: menyiapkan modul yang diperlukan, mengambil empat sensor dan objek point cloud, lalu membuat pemindai VLP-16 dengan putaran 5 Hz. Tampilan titik diatur agar menunjukkan titik terbaru. Data awal diisi nol agar Python tidak membaca data yang kosong.
2. sysCall_sensing: pada setiap langkah simulasi, script mengambil hasil pemindaian dalam koordinat dunia, lalu menyusunnya menjadi nomor paket, waktu simulasi, jumlah titik, dan koordinat XYZ setiap titik.
3. Pengiriman data: paket tersebut dikemas dan ditulis ke scene dengan nama properti di atas. Python membacanya melalui ZeroMQ dan memakai nomor paket agar data yang sama tidak terbaca dua kali.
4. sysCall_cleanup: menghapus pemindai ketika simulasi berakhir.

Program Python memeriksa jumlah data pada setiap paket supaya paket yang rusak tidak masuk ke peta. Frekuensi putaran 5 Hz berbeda dengan frekuensi pembacaan, karena pembacaan dilakukan pada setiap langkah simulasi.

4 Program Python dan File Keluaran

4.1 01_mapping.py: akuisisi peta

Program ini menghubungkan Python ke CoppeliaSim melalui localhost port 23000, dan simulasi harus berada pada kondisi Stop sebelum program dijalankan. Objek base dan target diambil lebih dulu sebelum simulasi dimulai karena target dilepas dari parent-nya oleh pengendali drone. Setelah hover awal, drone naik ke ketinggian terbang, menghadap arah gerak, lalu mengikuti waypoint.

Pada setiap langkah, program mencatat posisi drone, memeriksa status simulasi, dan membaca paket LiDAR. Bila posisi drone menyimpang lebih dari 1,5 m dari target, misi dihentikan. Titik dari paket baru disaring sesuai batas wilayah lalu dikumpulkan. Setelah misi selesai, warna Viridis dihitung dari ketinggian titik. Bila misi dihentikan di tengah jalan, data yang sudah terkumpul tetap disimpan dengan penanda parsial.

- point_cloud_3d_selesai.npz: berisi titik XYZ, trajectory, waypoint, status selesai, dan warna ketinggian. File ini menjadi masukan program 02.
- trajectory_3d_selesai.csv: catatan waktu dan koordinat XYZ drone yang digunakan untuk menganalisis jalur.
- Bila misi tidak selesai, akhiran selesai pada kedua nama file diganti dengan parsial.

Semua file disimpan pada folder hasil_mapping. Grafik Matplotlib menampilkan peta dan trajectory. Agar tampilan tidak berat, titik yang digambar dibatasi hingga 120.000 sampel, sedangkan seluruh titik tetap tersimpan pada file NPZ.

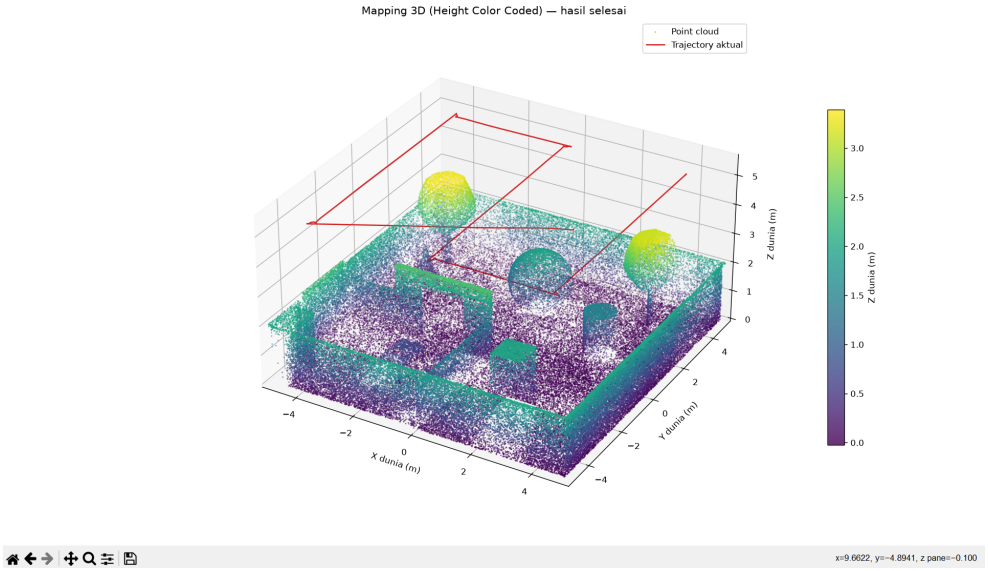


Figure 3: Pratinjau hasil mapping dari dataset 8.618.754 titik; grafik program 01 menampilkan maksimum 120.000 sampel beserta trajectory.

4.2 02_voxel_downsample.py: penyederhanaan titik

Program ini membaca file point_cloud_3d_selesai.npz, membuang koordinat yang tidak valid, lalu membagi ruang menjadi kotak-kotak kecil atau voxel berukuran 0,05 m. Semua titik yang berada dalam satu voxel diganti dengan satu titik rata-rata. Pemrosesan dilakukan per blok sebanyak 200.000 titik. Cara ini mengurangi pengukuran berulang pada permukaan yang sama, sementara bentuk umum lingkungan tetap terjaga. Implementasinya memakai NumPy dan bukan Open3D, dan hasilnya berupa titik perwakilan setiap voxel, bukan mesh atau kubus padat.

- point_cloud_voxel_5cm.npz: berisi seluruh titik hasil voxel beserta informasi pendukungnya, yaitu ukuran voxel, jumlah titik sumber, trajectory, waypoint, warna ketinggian, serta sampel 12.000, 25.000, dan 50.000 titik. File ini menjadi masukan program 03.
- point_cloud_12rb.npz, point_cloud_25rb.npz, dan point_cloud_50rb.npz: versi sampel yang lebih ringan untuk visualisasi atau dipakai secara terpisah. Jumlah titik sebenarnya mengikuti jumlah titik yang tersedia.

Program juga menampilkan tiga grafik perbandingan sampel. Karena file gambar tidak tersimpan otomatis, gambar pada laporan diambil melalui screenshot.

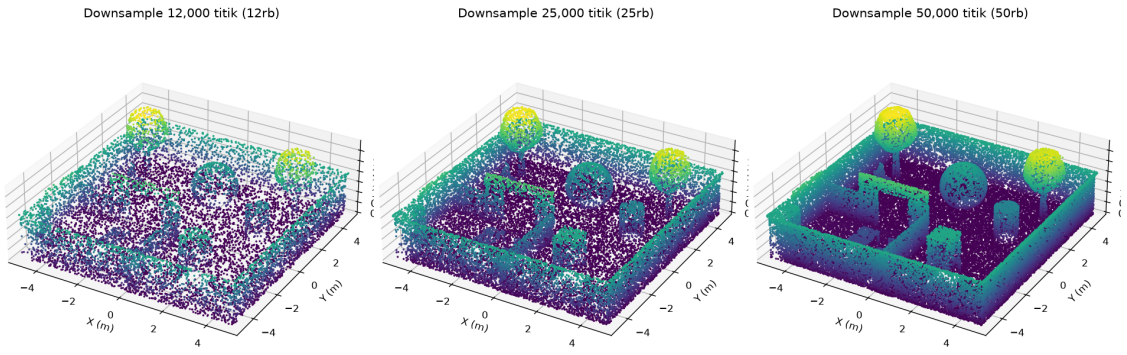


Figure 4: Perbandingan pratinjau 12.000, 25.000, dan 50.000 titik setelah voxel downsampling.

4.3 03_html_viewer.py: warna dan pengelompokan titik

Program ini membaca data voxel, menghitung warna ketinggian, dan mengelompokkan titik berdasarkan kedekatan posisinya. Titik dengan $z \leq 0{,}15$ m dianggap sebagai tanah. Titik lain yang berada pada $|x| \geq 4{,}85$ m atau $|y| \geq 4{,}85$ m dianggap sebagai batas scene dan dipisahkan. Sisanya dimasukkan ke dalam grid berukuran 0,10 m, lalu sel yang saling bersebelahan, termasuk yang bersebelahan secara diagonal, digabung menjadi satu kelompok. Kelompok dengan sedikitnya 60 titik diterima sebagai kelompok yang valid.

Setiap kelompok diberi nomor berdasarkan jumlah titiknya, tanpa mengenali nama bendanya. Dua benda yang bersentuhan dapat menyatu, sedangkan satu benda yang terukur secara terputus dapat terpecah. Parameter batas scene dibuat untuk lingkungan 10×10 m ini sehingga perlu disesuaikan bila scene berubah.

Warna ditentukan dari ketinggian titik, mulai dari yang terendah hingga tertinggi, kemudian dipetakan ke skala warna Viridis. File mapping_viewer_template.html harus berada satu folder dengan program. Python memasukkan data ke template tersebut sehingga menghasilkan satu file HTML yang dapat dibuka langsung di browser.

- mapping_3d_interaktif.html: tampilan hasil pemetaan dengan warna ketinggian, pilihan kelompok, trajectory, dan detail setiap kelompok.
- point_cloud_height_color.npz: data voxel beserta warna dan label kelompok.
- cluster_summary.json: ringkasan parameter pengelompokan serta jumlah dan rentang koordinat setiap kelompok.

Agar tampilan tetap ringan, peta utama pada HTML dibatasi maksimum 50.000 titik dan detail setiap kelompok maksimum 8.000 titik, sedangkan seluruh titik tetap tersimpan pada file NPZ. Tampilan viewer dibuat dengan gaya brutalism dan memakai Canvas sehingga tidak bergantung pada WebGL.

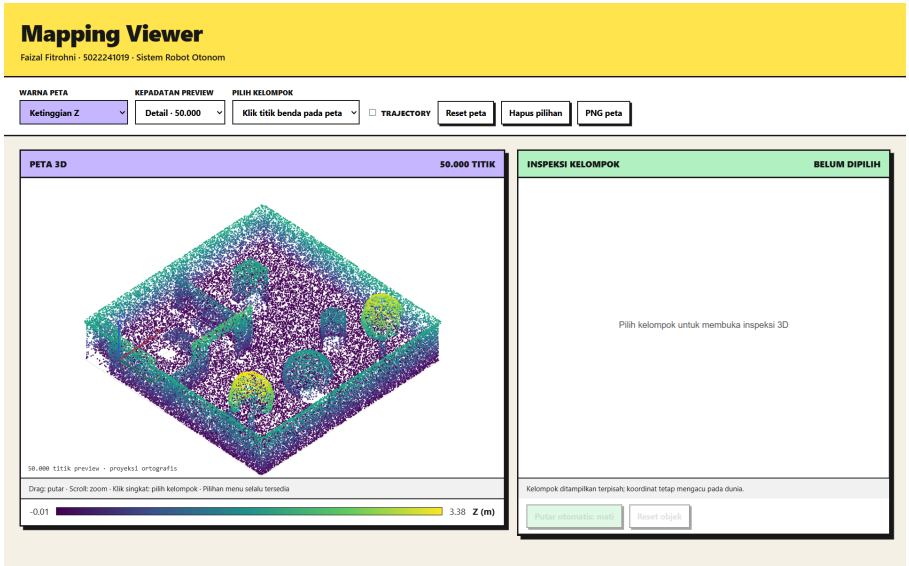


Figure 5: Viewer HTML hasil pemetaan dengan warna berdasarkan ketinggian dan informasi kelompok.

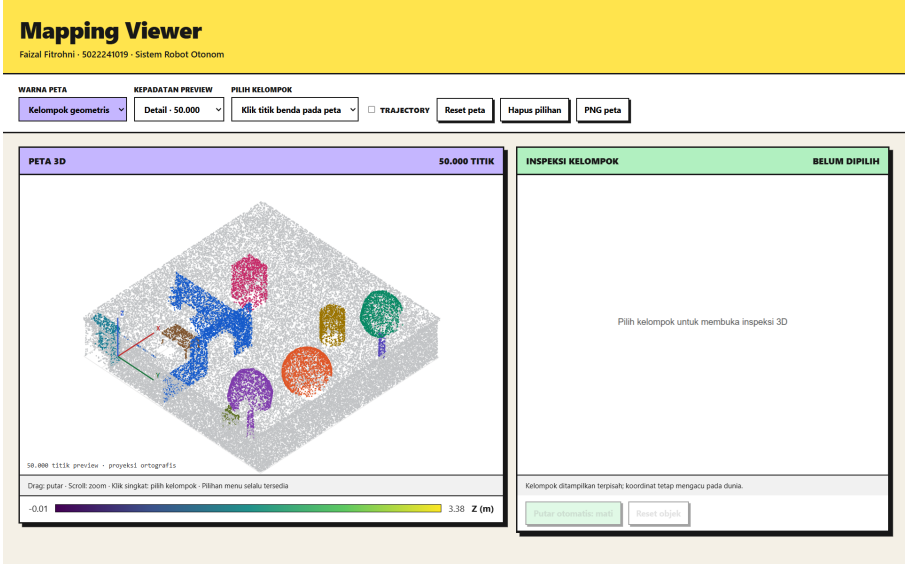


Figure 6: Pemilihan kelompok titik dan tampilan detail objek pada viewer HTML.

5 Hasil dan Analisis

Pada pengujian yang dicatat, dataset awal berisi 8.618.754 titik. Setelah voxel 5 cm, jumlahnya menjadi 172.925 titik sehingga pengurangannya sebesar

$$\left(1 - \frac{172925}{8618754}\right) \times 100\% = 97{,}99\%.$$

Pada pengujian voxel sebelumnya, waktu proses tercatat 16,6 s dan ukuran file turun dari 88,30 MiB menjadi 2,52 MiB. Angka ukuran file ini dapat berbeda pada program final karena ada tambahan warna dan sampel.

Penyederhanaan titik membuat visualisasi lebih ringan, tetapi detail yang lebih kecil dari ukuran voxel dapat berkurang. Warna ketinggian memudahkan pembedaan antara permukaan yang rendah dan yang tinggi. Warna ini menunjukkan nilai Z pada koordinat dunia, bukan warna asli objek.

Keterbatasan utama sistem ini adalah bagian objek yang tertutup, posisi drone yang masih diambil dari simulator, dan pengelompokan yang hanya berdasarkan kedekatan titik. Kelompok titik tidak dapat dianggap sebagai identitas objek. Bila objek baru ditambahkan dan seluruh proses dijalankan ulang, kelompok dihitung kembali sehingga nomornya dapat berubah. Simulasi ini juga belum mencakup kalibrasi sensor fisik, ketidakpastian posisi, maupun penghindaran rintangan otomatis.

6 Kesimpulan

Sistem yang dikembangkan menggabungkan kontrol target drone, pengambilan data LiDAR, penyimpanan trajectory dan point cloud, voxel downsampling, serta pewarnaan berdasarkan ketinggian. Pada pengujian, voxel 5 cm mengurangi 8.618.754 titik menjadi 172.925 titik. Viewer HTML memberikan cara yang interaktif untuk memeriksa peta dan kelompok titik tanpa machine learning. Hasil ini menunjukkan alur pemetaan dan pengolahan data dalam simulasi, dengan cakupan pengamatan dan pemisahan kelompok yang tetap dipengaruhi oleh posisi sensor, bagian yang tertutup, dan parameter pengolahan.